

Guida d'uso

Riprodurre il dimero senza rumore (Parte 1) in locale

Pacchetto riproducibile — Tesi triennale, Samuele Schiavi

Relatore: Prof. Alessandro Chiesa

Questo pacchetto contiene tutto il codice necessario per riprodurre, da zero, la Parte 1 (sistema chiuso, nessun rumore) del lavoro sul dimero: diagonalizzazione esatta, confronto fra ansatz VQE, evoluzione di Trotter e correlatori dinamici. E' il pacchetto *fratello* di `pacchetto_dimero_rumoroso_riproducibile` (Parte 2) — questo copre solo il sistema chiuso, non il rumore. Ogni modulo e ogni funzione sono commentati esplicitando sia il loro ruolo all'interno del proprio file sia il loro ruolo nell'insieme della pipeline.

Indice

1	Struttura del pacchetto	1
1.1	I 9 moduli di <code>codice/</code>	2
2	Prerequisiti	2
3	Il modo più semplice: un solo comando	2
4	Passo per passo, con la mappa esatta delle dipendenze	3
4.1	Onda 1 — nessuna dipendenza (possono girare in qualunque ordine, anche in parallelo)	3
4.2	Onda 2 — dipende dall'Onda 1	4
4.3	In sintesi: un solo ordine realmente obbligato	4
4.4	Tutti i comandi, in un unico blocco (nell'ordine di <code>esegui_tutto.py</code>)	5
5	Aggiornamento: figure aggiunte	5
6	Un risultato genuino scoperto riproducendo la figura di confronto	6
7	Come sapere se un numero è quello giusto	6
8	Relazione con il pacchetto rumoroso	6

1 Struttura del pacchetto

<code>codice/</code>	9 moduli Python (mattoni + confronto ansatz + Trotter + correlatori)
<code>dati/</code>	dati intermedi (<code>.npz</code>) generati da <code>01_</code> e <code>02_</code>
<code>figure/</code>	18 figure finali, in PDF e PNG (14 aggiunte in questo aggiornamento, Sez. 5)
<code>genera_figure/</code>	8 script (4 originali + 4 aggiunti)
<code>01_genera_dati_vqe_ideale.py</code>	script di generazione dati (VQE ideale)
<code>02_scan36_correlatori.py</code>	script di generazione dati (scan36, aggiunto)
<code>03_correlatore_da_vqe.py</code>	presente ma NON eseguito da <code>esegui_tutto.py</code> — problema aperto, Sez. 5
<code>esegui_tutto.py</code>	esegue tutto in un solo comando

I 9 moduli di codice/

File	Cosa fa
dimer_exact.py	Hamiltoniana e diagonalizzazione esatta — il “mattone zero”
vqe_test2.py	ansatz PMA-2q.3 e ottimizzazione VQE al punto di lavoro “test 2”
ansatz_dimero.py	libreria di varianti di ansatz (RBS, HA, PMA a diverse parametrizzazioni)
vqe_dimer.py	confronto sistematico HA vs PMA su tutto lo sweep in B/J
trotter_dimero.py	evoluzione di Trotter per decomposizione esplicita in gate nativi
circuito_correlazioni_dimero.py	circuito di Hadamard test per i correlatori dinamici
noise_model_dimero.py	incluso solo per la costante BASIS_GATES (vedi nota sotto)
stile.py	stile grafico condiviso da tutti gli script di figura
validate_ansatz_params_dimero.py	controllo obbligatorio: preparazione VQE vs ampiezze esatte nei correlatori
validate_circuito_correlazioni_dimero.py	controllo obbligatorio: correlatori contro riferimento classico, convergenza di Trotter

Una dipendenza reale verso il pacchetto rumoroso

`validate_ansatz_params_dimero.py` importa `BASIS_GATES` da `noise_model_dimero.py` — un modulo concettualmente di Parte 2. E' incluso qui **solo** per quella costante (la base di gate $\{rz, sx, x, cx\}$ usata per transpilare), non per la sua funzionalità di modellazione del rumore, mai usata in questo pacchetto.

2 Prerequisiti

Stessi di `pacchetto_dimero_rumoroso_riproducibile`:

Python	3.12.3
qiskit	2.5.2
qiskit-aer	0.17.2
numpy	2.4.4
scipy	1.17.1
matplotlib	3.10.8

Nessuna installazione LaTeX richiesta

Le figure sono generate direttamente in PDF/PNG da matplotlib.

3 Il modo più semplice: un solo comando

```
cd pacchetto_dimero_riproducibile
python3 esegui_tutto.py
```

Esegue in ordine: generazione dati (1 script), validazioni (2 script), tutte le 4 figure. Si ferma al primo errore.

Tempo stimato

1–2 minuti. La fase più lenta è la figura di confronto ansatz (circa 30 secondi per 80 ottimizzazioni VQE in tutto).

Se tutto va a buon fine, l'ultima riga stampata è:

```
TUTTO COMPLETATO. 4 figure in figure/, dati in dati/.
```

4 Passo per passo, con la mappa esatta delle dipendenze

Questa sezione non è solo un elenco di comandi: per ogni script indica cosa prende in ingresso, cosa produce (file e contenuto), e chi consuma quel prodotto più avanti nella catena. La buona notizia per questo pacchetto: la catena è quasi piatta. Un solo script produce dati; tre dei quattro script rimanenti (una validazione, tutte le figure) **non hanno bisogno** di quel file — calcolano da soli tutto ciò che serve, in isolamento.

Onda 1 — nessuna dipendenza (possono girare in qualunque ordine, anche in parallelo)

01_genera_dati_vqe_ideale.py

Ingresso: nessuno (parte da zero).

Uscita: `dati/ground_state_test2.npz` — contiene `b`, `J`, `D` (il punto di lavoro), `E0_exact`, `psi0_exact` (benchmark esatto), `vqe_params` (i 3 parametri ottimali dell'ansatz PMA-2q.3), `fidelity`, `E_vqe`.

Chi lo consuma: `validate_ansatz_params_dimero.py` (Onda 2, unico consumatore in questo pacchetto). Nota: è lo stesso file prodotto in `pacchetto_dimero_rumoroso_riproducibile` — se lo hai già generato là, puoi copiarlo qui invece di rieseguire lo script.

Valore atteso: energia ≈ -3.57321145 , fedeltà ≈ 1.0000000000 .

Comando (dalla radice del pacchetto):

```
python3 01_genera_dati_vqe_ideale.py
```

validate_circuito_correlazioni_dimero.py

Ingresso: nessuno — ricalcola da solo lo stato fondamentale via `ground_state()` di `circuito_correlazioni_dimero.py`.

Uscita: nessun file, solo un resoconto a schermo.

Chi lo consuma: nessuno (è un controllo terminale della catena).

Comando (dalla radice del pacchetto, nota il `cd`):

```
cd codice
python3 validate_circuito_correlazioni_dimero.py
cd ..
```

genera_fig_spettro_esatto.py

Ingresso: nessuno — diagonalizza da zero (nessun `.npz` richiesto).

Uscita: `figure/fig_spettro_dimero.pdf` (+`.png`).

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

```
python3 genera_figure/genera_fig_spettro_esatto.py
```

genera_fig_confronto_ansatz.py

Ingresso: nessuno — esegue da zero 80 ottimizzazioni VQE (4 configurazioni $\times$ 20 punti in `b`).

Uscita: `figure/fig_confronto_ansatz_HA_PMA.pdf` (+`.png`).

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto, ~ 30 secondi):

```
python3 genera_figure/genera_fig_confronto_ansatz.py
```

`genera_fig_trotter_convergenza.py`

Ingresso: nessuno — costruisce l'evoluzione via `trotter_dimero.py` da zero, a b, J, D fissati nello script.

Uscita: `figure/fig_trotter_convergenza.pdf (+.png)`.

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

```
python3 genera_figure/genera_fig_trotter_convergenza.py
```

`genera_fig_correlatore_convergenza.py`

Ingresso: nessuno — ricalcola lo stato fondamentale e il riferimento classico da zero.

Uscita: `figure/fig_correlatore_convergenza.pdf (+.png)`.

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

```
python3 genera_figure/genera_fig_correlatore_convergenza.py
```

Onda 2 — dipende dall'Onda 1

`validate_ansatz_params_dimero.py`

Ingresso: `ground_state_test2.npz` (copiato in codice/) — legge `vqe_params` e `fidelity`.

Uscita: nessun file, solo un resoconto a schermo (include il controllo 18 vs 19 CNOT).

Chi lo consuma: nessuno (controllo terminale).

Comando (dalla radice del pacchetto, dopo aver eseguito `01_`):

```
cp dati/ground_state_test2.npz codice/
cd codice
python3 validate_ansatz_params_dimero.py
cd ..
```

In sintesi: un solo ordine realmente obbligato

L'unica dipendenza vera di questo pacchetto

`01_genera_dati_vqe_ideale.py` → `validate_ansatz_params_dimero.py`. Tutto il resto (l'altra validazione, tutte e 4 le figure) è indipendente e può girare prima, dopo, o senza mai eseguire `01_` — `esegui_tutto.py` li esegue comunque tutti in un ordine fisso solo per chiarezza di lettura del log, non perché serva.

Tutti i comandi, in un unico blocco (nell'ordine di esegui_tutto.py)

```
cd pacchetto_dimero_riproducibile
python3 01_genera_dati_vqe_ideale.py
python3 02_scan36_correlatori.py
cp dati/*.npz codice/
cd codice
python3 validate_ansatz_params_dimero.py
python3 validate_circuito_correlazioni_dimero.py
cd ..
python3 genera_figure/genera_fig_confronto_ansatz.py
python3 genera_figure/genera_fig_correlatore_convergenza.py
python3 genera_figure/genera_fig_spettro_esatto.py
python3 genera_figure/genera_fig_trotter_convergenza.py
python3 genera_figure/genera_fig_gap_magnetizzazione.py
python3 genera_figure/genera_fig_vqe_riparazione.py
python3 genera_figure/genera_fig_trotter_circuito_vs_matrice.py
python3 genera_figure/genera_fig_scan36.py
python3 genera_figure/genera_circuiti.py
```

Risultati chiave dei due controlli

`validate_ansatz_params_dimero.py`: costo in gate, preparazione esatta 18 CNOT contro preparazione VQE 19 CNOT (coincide esattamente con quanto atteso).
`validate_circuito_correlazioni_dimero.py`: convergenza dell'errore di Trotter sul correlatore, rapporto che tende a 2 raddoppiando N (Trotter al prim'ordine, errore $O(1/N)$).

Gli script di figura vanno eseguiti dalla **radice** del pacchetto, non da dentro `genera_figure/`.

5 Aggiornamento: figure aggiunte

Rispetto alla prima versione (4 figure, nessun diagramma di circuito), aggiunte 14 figure per allineare il pacchetto al contenuto reale dei quattro documenti narrativi (`tesi_dimero_slides`):

- **Il gap esplicito** (`genera_fig_gap_magnetizzazione.py`): distanza fondamentale-primo eccitato contro B/J , non solo lo spettro completo — minimo verificato 0.565 con $D/J = 0.2$, coincidente.
- **La riparazione PMA** (`genera_fig_vqe_riparazione.py`): PMA-2q.3 (3 parametri) raggiunge $\mathcal{F} > 0.9998$ su tutto lo sweep, contro il crollo a ≈ 0.82 di PMA a $K = 1$ (Sez. 6 sotto).
- **Circuito vero contro matrice compatta** (`genera_fig_trotter_circuito_vs_matrice.py`): la prima versione usava solo la matrice compatta (`U_trotter_mat`) come unico metodo; qui entrambi i metodi sono calcolati e sovrapposti, verificato che coincidono a precisione macchina ($\sim 10^{-14}$), al punto di lavoro $b/J = -0.18$, $D/J = 1$ (stato $|00\rangle$), deliberatamente asimmetrico — vedi il documento per il perché).
- **Scan36** (`02_scan36_correlatori.py` + `genera_fig_scan36.py`): tutte le 36 combinazioni correlatore, verificato 12/36 zeri a $t = 0$, 0/36 zeri strutturali (nessuno resta nullo per $\max_t$).
- **Tutti i nove diagrammi di circuito** (`genera_circuiti.py`): assenti per scelta esplicita dell'autore originale (vedi docstring di `ansatz_dimero.py`). Cinque riusano le chiamate esatte di un vecchio script del progetto (`fig_circuiti.py`, non incluso in questo pacchetto, letto come riferimento); i restanti quattro (`circ_pma_base_ramo2`, `circ_rbs_espanso`, `circ_w_espanso`, `circ_cnotry`) non avevano uno script sorgente disponibile — costruiti direttamente dalle definizioni dei blocchi e dalle didascalie dei documenti.

Problema aperto, non risolto: fedeltà di PMA base a $b/J = -0.18$, $D/J = 1$

Il Documento 2 (§6, tabella di confronto) riporta $\mathcal{F} = 0.922857$ per l'ansatz PMA base (1 parametro) a questo punto di lavoro. La ricostruzione più diretta di questo ansatz (`ansatz_PMA_W/make_pma_ansatz`, verificate identiche fra loro) raggiunge invece un tetto di $\mathcal{F} = 0.851664$, verificato con: scan fine su θ (non un problema di ottimizzatore); minimizzazione dell'energia invece della fedeltà diretta (stesso risultato); scambio della convenzione qubit (nessuna differenza); direzione del CNOT e qubit di applicazione della R_y scambiati (nessuna variante supera ≈ 0.85); $D = 0.2$ invece di $D = 1$ nello stesso punto ($\mathcal{F} = 0.990$, ancora diverso da 0.922857); segno di D invertito (identico, il sistema è simmetrico per $D \rightarrow -D$); segno di b invertito (identico, simmetrico anche per $b \rightarrow -b$); $b = 0$ esatto ($\mathcal{F} = 0.853553$, praticamente indistinguibile da $b = -0.18$ — non è un problema di precisione su b). Il valore di D che darebbe davvero 0.922857 è $D \approx 0.62$ – 0.63 : non un valore “pulito”, non coincide né con $D = 1$ né con $D = 0.2$. Causa non identificata dopo nove ipotesi testate — probabilmente non risolvibile senza accesso al codice/sessione originale che ha prodotto il numero 0.922857 . **La figura `fig14_correlatore_da_vqe` non è stata prodotta** in attesa di risolvere questa discrepanza. Lo script `03_correlatore_da_vqe.py` esiste nel pacchetto ma non è richiamato da `esegui_tutto.py` per questo motivo.

Non ancora affrontate in questo aggiornamento: `fig09` (ricco/piatto), `fig10/fig13` (ricostruzione spettrale), `fig15/fig16` (scan36 con circuito VQE reale, costoso computazionalmente).

6 Un risultato genuino scoperto riproducendo la figura di confronto

PMA a $K = 1$ non è sempre sufficiente

Nella figura `fig_confronto_ansatz_HA_PMA`, la riga “PMA $D = 0.2$ ” mostra un calo netto di fedeltà (minimo ≈ 0.82 , non 1.0) proprio nella zona di anticrossing ($B/J \approx 2$) — a differenza di HA, che raggiunge sempre precisione macchina. Non è un bug: con un solo parametro e uno stato iniziale scelto in modo rigido (singoletto o $|11\rangle$ a seconda di B/J), l'ansatz PMA più semplice non riesce a interpolare con continuità attraverso il crossover indotto da $D \neq 0$. E' precisamente il motivo per cui il resto del progetto usa un ansatz PMA più espressivo (PMA-2q.3, 3 parametri, che rompe la conservazione di M) invece di questa versione a un solo parametro.

7 Come sapere se un numero è quello giusto

Ogni script stampa a schermo il valore appena calcolato **accanto** al valore atteso.

Se un numero non coincide

Qualcosa nel tuo ambiente locale sta producendo un risultato diverso: fermati e confronta prima di proseguire.

8 Relazione con il pacchetto rumoroso

Stessa disciplina, stesso punto di partenza

Questo pacchetto e `pacchetto_dimero_rumoroso_riproducibile` condividono lo script `01_genera_dati_vqe_ideale.py` e tre moduli (`dimer_exact.py`, `vqe_test2.py`, `circuito_correlazioni_dimero.py`) — non per caso: la Parte 2 (rumore) parte esattamente da dove finisce questa Parte 1, riusando lo stesso punto di lavoro e la stessa preparazione VQE. Se hai già eseguito il pacchetto rumoroso, `dati/ground_state_test2.npz` è identico in entrambi.

Verificato end-to-end, non solo assemblato

L'intera pipeline è stata rieseguita da zero (cartelle `dati/` e `figure/` svuotate, poi `esegui_tutto.py` rilanciato) dopo ogni modifica sostanziale al codice — ogni numero riportato in questa guida è stato ricontrollato. Verificato di nuovo dopo l'aggiornamento di Sez. 5: `exit code 0`, 18 figure prodotte (esclusa `fig14_correlatore_da_vqe`, non ancora prodotta per il problema aperto documentato sopra).